

First Principles of Deep Learning

Steve Avsec

Illinois Institute of Technology

March 6, 2025

Overview

- 1 A Little Information Theory
- 2 Overview
- 3 Deep Learning Architectures
 - Recurrent Neural Networks (RNNs)
 - Regularization of DL Models
 - Optimization

Entropy

Entropy is a measure of disorder or uncertainty in a system.

Entropy

Entropy is a measure of disorder or uncertainty in a system.

For a probability distribution, p :

$$H(p) = -E_p[\log(X)] = -\sum_{j=1}^n p_j \log(p_j)$$

Entropy

Entropy is a measure of disorder or uncertainty in a system.

For a probability distribution, p :

$$H(p) = -E_p[\log(X)] = -\sum_{j=1}^n p_j \log(p_j)$$

Maximized when p is the uniform distribution.

KL Divergence and Cross-Entropy

Let p and q be two probability distributions. Then

$$D_{KL}(p||q) = E_p[\log(\frac{p(X)}{q(X)})] = \sum_{j=1}^n p_j \log(p_j) - \sum_{j=1}^n p_j \log(q_j)$$

KL Divergence and Cross-Entropy

Let p and q be two probability distributions. Then

$$D_{KL}(p||q) = E_p[\log(\frac{p(X)}{q(X)})] = \sum_{j=1}^n p_j \log(p_j) - \sum_{j=1}^n p_j \log(q_j)$$

KL Divergence is a measure of "surprise": If q is the assumed distribution and p is the actual, $D_{KL}(p||q)$ can be interpreted as a difference between these realities.

KL Divergence and Cross-Entropy

Let p and q be two probability distributions. Then

$$D_{KL}(p||q) = E_p[\log(\frac{p(X)}{q(X)})] = \sum_{j=1}^n p_j \log(p_j) - \sum_{j=1}^n p_j \log(q_j)$$

KL Divergence is a measure of "surprise": If q is the assumed distribution and p is the actual, $D_{KL}(p||q)$ can be interpreted as a difference between these realities.

The *cross-entropy* of two distributions p and q is defined by

$$H(p, q) = -E_p[\log(q(X))] = -\sum_{j=1}^n p_j \log(q_j)$$

KL Divergence and Cross-Entropy

Let p and q be two probability distributions. Then

$$D_{KL}(p||q) = E_p[\log(\frac{p(X)}{q(X)})] = \sum_{j=1}^n p_j \log(p_j) - \sum_{j=1}^n p_j \log(q_j)$$

KL Divergence is a measure of "surprise": If q is the assumed distribution and p is the actual, $D_{KL}(p||q)$ can be interpreted as a difference between these realities.

The *cross-entropy* of two distributions p and q is defined by

$$H(p, q) = -E_p[\log(q(X))] = -\sum_{j=1}^n p_j \log(q_j)$$

Often, we want to minimize $D_{KL}(p||q)$, but because

$$H(p, q) = H(p) + D_{KL}(p||q)$$

we often minimize $H(p, q)$.

Maximum Likelihood Estimation

Let $\mathbf{X}$ be a data set drawn from some unknown probability distribution p_{data}

Maximum Likelihood Estimation

Let $\mathbf{X}$ be a data set drawn from some unknown probability distribution p_{data}

Let $p_{model}(\mathbf{x}, \theta)$ be a parameterized family of models.

Maximum Likelihood Estimation

Let $\mathbf{X}$ be a data set drawn from some unknown probability distribution p_{data}

Let $p_{model}(\mathbf{x}, \theta)$ be a parameterized family of models.

$p_{model}(\mathbf{x}, \theta)$ is an estimate to $p_{data}(\mathbf{x})$ for a fixed θ .

MLE Continued

$$\begin{aligned}\theta_{ML} &= \operatorname{argmax}_{\theta} p_{\text{model}}(\mathbf{X}, \theta) \\ &= \operatorname{argmax}_{\theta} \prod_{j=1}^N p_{\text{model}}(\mathbf{x}_j, \theta) \\ &= \operatorname{argmax}_{\theta} \max_{\theta} \sum_{j=1}^N \log(p_{\text{model}}(\mathbf{x}, \theta)) \\ &= \operatorname{argmax}_{\theta} \max_{\theta} E_{\hat{p}_{\text{data}}} \log(p_{\text{model}}(\mathbf{x}))\end{aligned}$$

MLE Continued

$$\begin{aligned}\theta_{ML} &= \operatorname{argmax}_{\theta} p_{model}(\mathbf{X}, \theta) \\ &= \operatorname{argmax}_{\theta} \prod_{j=1}^N p_{model}(\mathbf{x}_j, \theta) \\ &= \operatorname{argmax}_{\theta} \max \sum_{j=1}^N \log(p_{model}(\mathbf{x}, \theta)) \\ &= \operatorname{argmax}_{\theta} \max E_{\hat{p}_{data}} \log(p_{model}(\mathbf{x}))\end{aligned}$$

and we can conclude that the maximum likelihood estimator θ_{ML} coincides with the minimizer of $H(p_{data}, p_{model})$

Linear Models vs. Deep Learning

Goal: Optimize

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

according to some loss function given a data set $(\mathbf{X}, \mathbf{y})$.

Linear Models vs. Deep Learning

Goal: Optimize

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

according to some loss function given a data set $(\mathbf{X}, \mathbf{y})$.

Linear models:

$$f(\mathbf{x}) = \sum_{j=1}^K c_j f_j(\mathbf{x})$$

where $f_j \in \mathcal{F}$ and $\mathcal{F}$ is some appropriate class of functions.

Linear Models vs. Deep Learning

Goal: Optimize

$$f : \mathbb{R}^d \rightarrow \mathbb{R}$$

according to some loss function given a data set $(\mathbf{X}, \mathbf{y})$.

Linear models:

$$f(\mathbf{x}) = \sum_{j=1}^K c_j f_j(\mathbf{x})$$

where $f_j \in \mathcal{F}$ and $\mathcal{F}$ is some appropriate class of functions.

Deep learning:

$$f(\mathbf{x}) = f^{(K)} \circ \dots \circ f^{(2)} \circ f^{(1)}(\mathbf{x})$$

where $f^{(k)} : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^{d_{k+1}}$ and $d_{K+1} = 1$.

Some Jargon

- The model described here is a feedforward neural network.

Some Jargon

- The model described here is a feedforward neural network.
- The functions $f^{(k)}$ are often referred to as “hidden” layers.

Some Jargon

- The model described here is a feedforward neural network.
- The functions $f^{(k)}$ are often referred to as “hidden” layers.
- The *depth* of a neural network is the number of layers (K).

Some Jargon

- The model described here is a feedforward neural network.
- The functions $f^{(k)}$ are often referred to as "hidden" layers.
- The *depth* of a neural network is the number of layers (K).
- The *width* of a neural network is the maximum dimension $\max_{1 \leq k \leq K} d_k$.

Some building blocks

- Loss functions (also commonly called cost functions in the DL literature).

Some building blocks

- Loss functions (also commonly called cost functions in the DL literature).
- Activation functions
 - ReLU (rectified linear unit)

$$g(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

Some building blocks

- Loss functions (also commonly called cost functions in the DL literature).
- Activation functions
 - ReLU (rectified linear unit)

$$g(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

- Hyperbolic tangent

$$g(x) = \tanh(x)$$

Output Units

- Linear (Gaussian):

$$\mathbf{y} = \mathbf{W}^t \mathbf{h} + \mathbf{b}$$

Output Units

- Linear (Gaussian):

$$\mathbf{y} = \mathbf{W}^t \mathbf{h} + \mathbf{b}$$

- Sigmoid (Bernoulli):

$$y = \sigma(\mathbf{w}^t \mathbf{h} + b)$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$.

Output Units

- Linear (Gaussian):

$$\mathbf{y} = \mathbf{W}^t \mathbf{h} + \mathbf{b}$$

- Sigmoid (Bernoulli):

$$y = \sigma(\mathbf{w}^t \mathbf{h} + b)$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$.

- Softmax (Multinomial):

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$$

where $z_i = \log(y = i | \mathbf{h}) = \mathbf{W}^t \mathbf{h} + \mathbf{b}$

Gradients

Feedforward neural networks lose *convexity* of their loss function with respect to model parameters.

Gradients

Feedforward neural networks lose *convexity* of their loss function with respect to model parameters.

This loss of convexity means that solutions are sensitive to initial parameters and convergence is not guaranteed.

Gradients

Feedforward neural networks lose *convexity* of their loss function with respect to model parameters.

This loss of convexity means that solutions are sensitive to initial parameters and convergence is not guaranteed.

It is called *saturation* when a function becomes very flat and its gradient becomes close to 0.

Architecture

Layers are typically broken down by

$$\mathbf{h}^{(1)} = g^{(1)}(\mathbf{W}^{(1)t}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{h}^{(2)} = g^{(2)}(\mathbf{W}^{(2)t}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

et cetera.

Architecture

Layers are typically broken down by

$$\mathbf{h}^{(1)} = g^{(1)}(\mathbf{W}^{(1)t}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{h}^{(2)} = g^{(2)}(\mathbf{W}^{(2)t}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

et cetera.

Theorem (The Universal Approximation Theorem)

A feedforward neural network with at least one hidden layer and a linear output layer and any "reasonable" activation function can approximate any "reasonable" function.

Architecture

Layers are typically broken down by

$$\mathbf{h}^{(1)} = g^{(1)}(\mathbf{W}^{(1)t}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{h}^{(2)} = g^{(2)}(\mathbf{W}^{(2)t}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

et cetera.

Theorem (The Universal Approximation Theorem)

A feedforward neural network with at least one hidden layer and a linear output layer and any "reasonable" activation function can approximate any "reasonable" function.

The downside: While such a neural network can *represent* any function, due to optimization issues with non-convex functions, there is no guarantee that such a representation can be learned.

The Chain Rule

Let

- $\mathbf{x} \in \mathbb{R}^d$,
- $g : \mathbb{R} \rightarrow \mathbb{R}$ be some activation function,
- $\mathbf{W}$ be an d -dim vector,
- and b be some offset.

The Chain Rule

Let

- $\mathbf{x} \in \mathbb{R}^d$,
- $g : \mathbb{R} \rightarrow \mathbb{R}$ be some activation function,
- $\mathbf{W}$ be an d -dim vector,
- and b be some offset.

If

$$f(\mathbf{x}) = g(\mathbf{w}^t \mathbf{x} + b)$$

then

The Chain Rule

Let

- $\mathbf{x} \in \mathbb{R}^d$,
- $g : \mathbb{R} \rightarrow \mathbb{R}$ be some activation function,
- $\mathbf{W}$ be an d -dim vector,
- and b be some offset.

If

$$f(\mathbf{x}) = g(\mathbf{w}^t \mathbf{x} + b)$$

then

$$\nabla f(\mathbf{x}) = g'(\mathbf{w}^t \mathbf{x} + b) \times \mathbf{w}^t$$

Backprop

Let l denote the depth of the feedforward neural network.

① Let $\mathbf{g} = \nabla_{\hat{\mathbf{y}}} L(\mathbf{y}, \hat{\mathbf{y}})$.

Backprop

Let l denote the depth of the feedforward neural network.

- 1 Let $\mathbf{g} = \nabla_{\hat{\mathbf{y}}} L(\mathbf{y}, \hat{\mathbf{y}})$.
- 2 For $k = l, l - 1, \dots, 1$:

Backprop

Let l denote the depth of the feedforward neural network.

- 1 Let $\mathbf{g} = \nabla_{\hat{\mathbf{y}}} L(\mathbf{y}, \hat{\mathbf{y}})$.
- 2 For $k = l, l - 1, \dots, 1$:
 - 1 $\mathbf{g} = \nabla_{\mathbf{a}^{(k)}} L = \mathbf{g} \circ g'(\mathbf{a}^{(k)})$

Backprop

Let l denote the depth of the feedforward neural network.

- 1 Let $\mathbf{g} = \nabla_{\hat{\mathbf{y}}} L(\mathbf{y}, \hat{\mathbf{y}})$.
- 2 For $k = l, l - 1, \dots, 1$:
 - 1 $\mathbf{g} = \nabla_{\mathbf{a}^{(k)}} L = \mathbf{g} \circ g'(\mathbf{a}^{(k)})$
 - 2 Gradients wrt $\mathbf{b}^{(k)}$ and $\mathbf{W}^{(k)}$ can be expressed by
 - $\nabla_{\mathbf{b}^{(k)}} L = \mathbf{g} + \text{reg.}$
 - $\nabla_{\mathbf{W}^{(k)}} L = \mathbf{g} + \text{reg.}$
 - $\mathbf{g} = \mathbf{W}^{(k)t} \mathbf{g}$

What is a convolution?

Let $\mathbf{a}, \mathbf{b}$ be two bi-infinite sequences.

What is a convolution?

Let $\mathbf{a}, \mathbf{b}$ be two bi-infinite sequences.

Definition

The *convolution* of $\mathbf{a}$ and $\mathbf{b}$ is given by

$$(\mathbf{a} \star \mathbf{b})(n) = \sum_{m=-\infty}^{\infty} \mathbf{a}(m)\mathbf{b}(n-m)$$

What is a convolution?

Let **a**, **b** be two bi-infinite sequences.

Definition

The *convolution* of **a** and **b** is given by

$$(\mathbf{a} \star \mathbf{b})(n) = \sum_{m=-\infty}^{\infty} \mathbf{a}(m)\mathbf{b}(n-m)$$

Why? Typically, this is viewed as an “averaging” operation between **a** and **b**.

That Sounds Very 1-D

We can lift this up to two dimensions by defining

$$(\mathbf{a} \star \mathbf{b})(m, n) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} \mathbf{a}(i, j) \mathbf{b}((m, n) - (i, j))$$

That Sounds Very 1-D

We can lift this up to two dimensions by defining

$$(\mathbf{a} \star \mathbf{b})(m, n) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} \mathbf{a}(i, j) \mathbf{b}((m, n) - (i, j))$$

If we keep indexing this way we'll run out of letters so instead

$$(\mathbf{a} \star \mathbf{b})(\mathbf{n}) = \sum_{\mathbf{m} \in \mathbb{Z}^d} \mathbf{a}(\mathbf{m}) \mathbf{b}(\mathbf{n} - \mathbf{m})$$

where $d = 1, 2, 3, \dots$

Yeah, But Infinity Doesn't Exist

If we have two finite sequences we can

- 1 Pad zeros out to infinity.

Yeah, But Infinity Doesn't Exist

If we have two finite sequences we can

- 1 Pad zeros out to infinity.
- 2 Impose a *Dirichlet* type boundary

$$\mathbf{r}(\mathbf{m}) = c$$

for some constant c and $\mathbf{m}$ on the boundary.

Yeah, But Infinity Doesn't Exist

If we have two finite sequences we can

- 1 Pad zeros out to infinity.
- 2 Impose a *Dirichlet* type boundary

$$\mathbf{r}(\mathbf{m}) = c$$

for some constant c and $\mathbf{m}$ on the boundary.

- 3 Impose a *Neumann* type boundary

$$\partial \mathbf{r}(\mathbf{m}) = c$$

where ∂ is some difference operator and $\mathbf{m}$ is on the boundary and c is some constant.

What does this have to do with Deep Learning?

Convolutional layers lead to three positive attributes in feedforward architectures:

- 1 Sparse interactions.
 - Example: An image might have a million pixels, but the kernel might only look at a 10 by 10 window

What does this have to do with Deep Learning?

Convolutional layers lead to three positive attributes in feedforward architectures:

- 1 Sparse interactions.
 - Example: An image might have a million pixels, but the kernel might only look at a 10 by 10 window
- 2 Parameter sharing.
 - Example: You only have to learn parameters for the 100 parameter kernel in the previous example.

What does this have to do with Deep Learning?

Convolutional layers lead to three positive attributes in feedforward architectures:

- 1 Sparse interactions.
 - Example: An image might have a million pixels, but the kernel might only look at a 10 by 10 window
- 2 Parameter sharing.
 - Example: You only have to learn parameters for the 100 parameter kernel in the previous example.
- 3 Equivariance.
 - If you choose your kernel correctly, images can be rotated, translated, lightened, darkened and the output of the convolution will be the same.

Pooling

Pooling is an operation that takes several "nearby" inputs and produces an output via some summary statistic.

- Max pooling: Take the maximum of the related inputs.

Pooling

Pooling is an operation that takes several "nearby" inputs and produces an output via some summary statistic.

- Max pooling: Take the maximum of the related inputs.
- Average pooling

Pooling

Pooling is an operation that takes several "nearby" inputs and produces an output via some summary statistic.

- Max pooling: Take the maximum of the related inputs.
- Average pooling
- Median pooling

Typical Structure

A "typical" convolutional layer is composed of

- An affine transformation
 - Example: Difference by one shift to the left.

Typical Structure

A "typical" convolutional layer is composed of

- An affine transformation
 - Example: Difference by one shift to the left.
- A nonlinear function e.g. ReLU

Typical Structure

A "typical" convolutional layer is composed of

- An affine transformation
 - Example: Difference by one shift to the left.
- A nonlinear function e.g. ReLU
- A pooling operation

Recurrence Relations

Let $\{s_1, \dots, s_n\}$ be a set of states.

Definition

A recurrence relation is a function of the form

$$s_{t_j} = f(s_{t_j-1}, \theta)$$

where θ is some set of parameters

Recurrence Relations

Let $\{s_1, \dots, s_n\}$ be a set of states.

Definition

A recurrence relation is a function of the form

$$s_{t_j} = f(s_{t_j-1}, \theta)$$

where θ is some set of parameters

Could instead learn a function like

$$s_{t_j} = g(s_{t_j-1}, \dots, s_{t_1}, \theta)$$

but such a g can't handle flexible input lengths

Why use these things?

- Language modeling
 - Entity recognition
 - Sentiment analysis
 - Text summarization

Why use these things?

- Language modeling
 - Entity recognition
 - Sentiment analysis
 - Text summarization
- Transaction fraud detection
 - Input: A transaction history for a bunch of shoppers/shops.
 - Training set: A set of transactions labeling fraudulent transactions.
 - Classify most recent transactions given an arbitrary length history.

Why use these things?

- Language modeling
 - Entity recognition
 - Sentiment analysis
 - Text summarization
- Transaction fraud detection
 - Input: A transaction history for a bunch of shoppers/shops.
 - Training set: A set of transactions labeling fraudulent transactions.
 - Classify most recent transactions given an arbitrary length history.
- Time series prediction
 - LSTM (long-short term memory)

Three Design Choices

- Have an output at each time step and have recurrent connections between hidden units.

Three Design Choices

- Have an output at each time step and have recurrent connections between hidden units.
- Have an output at each time step and have recurrent connections only between output of the previous output and next hidden units.

Three Design Choices

- Have an output at each time step and have recurrent connections between hidden units.
- Have an output at each time step and have recurrent connections only between output of the previous output and next hidden units.
- Have connections between hidden units but only produce one output per sequence.

Gradients

We can write down a system:

- $\mathbf{a}_t = \mathbf{b} + W\mathbf{h}_{t-1} + U\mathbf{x}_t$
- $\mathbf{h}_t = g(\mathbf{a})$
- $\mathbf{o}_t = \mathbf{c} + V\mathbf{h}_t$
- $\mathbf{y}_t = d(\mathbf{o}_t)$

Gradients

We can write down a system:

- $\mathbf{a}_t = \mathbf{b} + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t$
- $\mathbf{h}_t = g(\mathbf{a})$
- $\mathbf{o}_t = \mathbf{c} + \mathbf{V}\mathbf{h}_t$
- $\mathbf{y}_t = d(\mathbf{o}_t)$

Here g is some non-linear activation function and d is some decision function (e.g. softmax).

Gradients

We can write down a system:

- $\mathbf{a}_t = \mathbf{b} + W\mathbf{h}_{t-1} + U\mathbf{x}_t$
- $\mathbf{h}_t = g(\mathbf{a})$
- $\mathbf{o}_t = \mathbf{c} + V\mathbf{h}_t$
- $\mathbf{y}_t = d(\mathbf{o}_t)$

Here g is some non-linear activation function and d is some decision function (e.g. softmax).

We can "unroll" these equations and write down the gradients involved to fit the model.

Bidirectional RNNs

Essentially, train two networks, one forward and one backward and ensemble results.

Bidirectional RNNs

Essentially, train two networks, one forward and one backward and ensemble results.

Why? Language problems, for instance, often have information after the current state that is relevant.

Bidirectional RNNs

Essentially, train two networks, one forward and one backward and ensemble results.

Why? Language problems, for instance, often have information after the current state that is relevant.

State-of-the-art in speech recognition, entity recognition, translation, part-of-speech tagging, etc.

The Problem of Long-Term Behavior

- Take away inputs and outputs and you just have

$$\mathbf{h}_n = W^n \mathbf{h}_0$$

- A matrix W can (almost) always be decomposed by

$$W = Q^{-1} \Lambda Q$$

(eigen-decomposition).

- If all of the eigenvalues have absolute value less than one, W^n tends to the zero matrix.

The Problem of Long-Term Behavior

- Take away inputs and outputs and you just have

$$\mathbf{h}_n = W^n \mathbf{h}_0$$

- A matrix W can (almost) always be decomposed by

$$W = Q^{-1} \Lambda Q$$

(eigen-decomposition).

- If all of the eigenvalues have absolute value less than one, W^n tends to the zero matrix.
- If any of the eigenvalues have absolute value greater than one, W^n explodes in magnitude.

A Refresher

- A model like

$$f(\mathbf{x}) = f^{(K)} \circ \dots \circ f^{(2)} \circ f^{(1)}(\mathbf{x})$$

where $f^{(k)} : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^{d_{k+1}}$ and $d_{K+1} = 1$ is called a feedforward neural network.

A Refresher

- A model like

$$f(\mathbf{x}) = f^{(K)} \circ \dots \circ f^{(2)} \circ f^{(1)}(\mathbf{x})$$

where $f^{(k)} : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^{d_{k+1}}$ and $d_{K+1} = 1$ is called a feedforward neural network.

- Each f_k refers to a "layer".

A Refresher

- A model like

$$f(\mathbf{x}) = f^{(K)} \circ \dots \circ f^{(2)} \circ f^{(1)}(\mathbf{x})$$

where $f^{(k)} : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^{d_{k+1}}$ and $d_{K+1} = 1$ is called a feedforward neural network.

- Each f_k refers to a "layer".
- The number of layers is called the depth.

Building blocks

- "Most" layers can be decomposed into a function like

$$f_k(\mathbf{h}) = g \circ A(\mathbf{h})$$

Building blocks

- "Most" layers can be decomposed into a function like

$$f_k(\mathbf{h}) = g \circ A(\mathbf{h})$$

- Where g is some non-linear function (ReLU, tanh, Softmax, sigmoid/logit).

Building blocks

- "Most" layers can be decomposed into a function like

$$f_k(\mathbf{h}) = g \circ A(\mathbf{h})$$

- Where g is some non-linear function (ReLU, tanh, Softmax, sigmoid/logit).
- And A is an *affine* transformation of the form

$$A(\mathbf{h}) = W^t \mathbf{h} + \mathbf{b}$$

Norm Penalties

- Like linear models, DL models can use ℓ_1 - or ℓ_2 -norm regularization.

Norm Penalties

- Like linear models, DL models can use ℓ_1 - or ℓ_2 -norm regularization.
- Unlike linear models, we can apply different penalties to different layers or just add one lump ℓ_1/ℓ_2 term at the end.

Norm Penalties

- Like linear models, DL models can use ℓ_1 - or ℓ_2 -norm regularization.
- Unlike linear models, we can apply different penalties to different layers or just add one lump ℓ_1/ℓ_2 term at the end.
- Like in the linear case, ℓ_1 tends to sparse solutions while ℓ_2 tends to minimize out-of-sample error.

Norm Penalties

- Like linear models, DL models can use ℓ_1 - or ℓ_2 -norm regularization.
- Unlike linear models, we can apply different penalties to different layers or just add one lump ℓ_1/ℓ_2 term at the end.
- Like in the linear case, ℓ_1 tends to sparse solutions while ℓ_2 tends to minimize out-of-sample error.
- Typically only regularize W and not $\mathbf{b}$ in the affine transformation.

Dataset Augmentation

Disclaimer: This is a technique your instructor has seen misapplied more than once.

- Limited number of samples? No problem; generate fake samples that “look” like your original ones.

Dataset Augmentation

Disclaimer: This is a technique your instructor has seen misapplied more than once.

- Limited number of samples? No problem; generate fake samples that “look” like your original ones.
- Classic example: shift, stretch, or rotate images in image classification.

Dataset Augmentation

Disclaimer: This is a technique your instructor has seen misapplied more than once.

- Limited number of samples? No problem; generate fake samples that “look” like your original ones.
- Classic example: shift, stretch, or rotate images in image classification.
- Must take care that your “faking” mechanism does not change the label or output.

Noise Injection

- Idea: Add small scale, normal noise to the weight matrices at some or all of the layers.

Noise Injection

- Idea: Add small scale, normal noise to the weight matrices at some or all of the layers.
- Effect: The winning model tends to “find” regions of parameter space where the parameters are relatively robust against small perturbations (e.g. small changes in parameters do not lead to large changes in the loss function.)

Noise Injection

- Idea: Add small scale, normal noise to the weight matrices at some or all of the layers.
- Effect: The winning model tends to “find” regions of parameter space where the parameters are relatively robust against small perturbations (e.g. small changes in parameters do not lead to large changes in the loss function.)
- This is a feature of DL models; for linear models, the noise will not have an effect on the optimal parameters.

Noise Injection

- Idea: Add small scale, normal noise to the weight matrices at some or all of the layers.
- Effect: The winning model tends to “find” regions of parameter space where the parameters are relatively robust against small perturbations (e.g. small changes in parameters do not lead to large changes in the loss function.)
- This is a feature of DL models; for linear models, the noise will not have an effect on the optimal parameters.
- Related: Add noise to the “label” part of the training data.

Semi-supervised Models

- Suppose you have some labeled samples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ and some unlabeled samples $\{\mathbf{x}_{n+1}, \dots, \mathbf{x}_N\}$.

Semi-supervised Models

- Suppose you have some labeled samples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ and some unlabeled samples $\{\mathbf{x}_{n+1}, \dots, \mathbf{x}_N\}$.
- Idea: build generative models of $P(\mathbf{x})$ and $P(\mathbf{x}, y)$ and a discriminative model $P(y|\mathbf{x})$.

Semi-supervised Models

- Suppose you have some labeled samples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ and some unlabeled samples $\{\mathbf{x}_{n+1}, \dots, \mathbf{x}_N\}$.
- Idea: build generative models of $P(\mathbf{x})$ and $P(\mathbf{x}, y)$ and a discriminative model $P(y|\mathbf{x})$.
- Additional feature information coming from the unlabeled $\mathbf{x}$ samples give an advantage in classifying the y values.

Semi-supervised Models

- Suppose you have some labeled samples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$ and some unlabeled samples $\{\mathbf{x}_{n+1}, \dots, \mathbf{x}_N\}$.
- Idea: build generative models of $P(\mathbf{x})$ and $P(\mathbf{x}, y)$ and a discriminative model $P(y|\mathbf{x})$.
- Additional feature information coming from the unlabeled $\mathbf{x}$ samples give an advantage in classifying the y values.
- Application: Named-entity recognition.

Early Stopping

- The loss function applied to a validation set tends to decrease after some iterations and then increase.

Early Stopping

- The loss function applied to a validation set tends to decrease after some iterations and then increase.
- Solution: Cache the parameters with the minimal validation loss.

Early Stopping

- The loss function applied to a validation set tends to decrease after some iterations and then increase.
- Solution: Cache the parameters with the minimal validation loss.
- Pretty much ubiquitous in DL now.

Sparsity

- We all know that ℓ_1 regularization leads to sparse parameters.

Sparsity

- We all know that ℓ_1 regularization leads to sparse parameters.
- Other approaches ensure that we can find sparse *representations* of the original inputs.

Sparsity

- We all know that ℓ_1 regularization leads to sparse parameters.
- Other approaches ensure that we can find sparse *representations* of the original inputs.
- Solve minimization problem:

$$\arg \min_{\mathbf{h}: \|\mathbf{h}\|_0 \leq k} \|\mathbf{x} - W\mathbf{h}\|_2^2$$

where $\|\mathbf{h}\|_0$ denotes the number of non-zero components

Matching basis pursuit

- Start with $\mathbf{x}$ and a random matrix with unit columns W .

Matching basis pursuit

- Start with $\mathbf{x}$ and a random matrix with unit columns W .
- Let $\mathbf{v}_1 = \mathbf{x}$ and then find the column $\mathbf{w}_{k_n}$ of W that maximizes

$$|\langle \mathbf{v}_n, \mathbf{w}_{k_n} \rangle|$$

Matching basis pursuit

- Start with $\mathbf{x}$ and a random matrix with unit columns W .
- Let $\mathbf{v}_1 = \mathbf{x}$ and then find the column $\mathbf{w}_{k_n}$ of W that maximizes

$$|\langle \mathbf{v}_n, \mathbf{w}_{k_n} \rangle|$$

- Let $a_n = \langle \mathbf{v}_n, \mathbf{w}_{k_n} \rangle$ and

$$\mathbf{v}_{n+1} = \mathbf{v}_n - a_n \mathbf{w}_{k_n}$$

Matching basis pursuit

- Start with $\mathbf{x}$ and a random matrix with unit columns W .
- Let $\mathbf{v}_1 = \mathbf{x}$ and then find the column $\mathbf{w}_{k_1}$ of W that maximizes

$$|\langle \mathbf{v}_n, \mathbf{w}_{k_n} \rangle|$$

- Let $a_n = \langle \mathbf{v}_n, \mathbf{w}_{k_n} \rangle$ and

$$\mathbf{v}_{n+1} = \mathbf{v}_n - a_n \mathbf{w}_{k_n}$$

- Stop if $\|\mathbf{v}_n\|_2$ is less than some threshold.

Bagging

- Idea: Similar to bootstrapping, sample k different training datasets by sampling from the original training data (with replacement).

Bagging

- Idea: Similar to bootstrapping, sample k different training datasets by sampling from the original training data (with replacement).
- Train model j on training set j .

Bagging

- Idea: Similar to bootstrapping, sample k different training datasets by sampling from the original training data (with replacement).
- Train model j on training set j .
- Ensemble the k resulting models by "voting" on the best outcome.

Bagging

- Idea: Similar to bootstrapping, sample k different training datasets by sampling from the original training data (with replacement).
- Train model j on training set j .
- Ensemble the k resulting models by "voting" on the best outcome.
- Downside: Computationally expensive for large networks.

Dropout

- Sample a minibatch from the training data and select a *mask* of the input features + hidden units.

Dropout

- Sample a minibatch from the training data and select a *mask* of the input features + hidden units.
- "Typical" selection is an input feature is included with probability 0.8 and hidden unit with probability 0.5.

Dropout

- Sample a minibatch from the training data and select a *mask* of the input features + hidden units.
- "Typical" selection is an input feature is included with probability 0.8 and hidden unit with probability 0.5.
- Parameters are saved and shared among each iteration.

Dropout

- Sample a minibatch from the training data and select a *mask* of the input features + hidden units.
- "Typical" selection is an input feature is included with probability 0.8 and hidden unit with probability 0.5.
- Parameters are saved and shared among each iteration.
- Even a relatively few draws of masks and few trainings can yield very good models.

Stochastic Gradient Descent

- 1 Select a sequence of learning rates (ε_k) and initial parameter set θ .

Stochastic Gradient Descent

- 1 Select a sequence of learning rates (ε_k) and initial parameter set θ .
- 2 Sample a small batch of the training data.

Stochastic Gradient Descent

- 1 Select a sequence of learning rates (ε_k) and initial parameter set θ .
- 2 Sample a small batch of the training data.
- 3 Compute

$$\mathbf{g} = \frac{1}{m} \nabla L(f(\mathbf{x}, \theta), y)$$

Stochastic Gradient Descent

- 1 Select a sequence of learning rates (ε_k) and initial parameter set θ .
- 2 Sample a small batch of the training data.

- 3 Compute

$$\mathbf{g} = \frac{1}{m} \nabla L(f(\mathbf{x}, \theta), y)$$

- 4 k th iteration,

$$\theta_{k+1} = \theta_k - \varepsilon_k \mathbf{g}$$

Stochastic Gradient Descent

- 1 Select a sequence of learning rates (ε_k) and initial parameter set θ .
- 2 Sample a small batch of the training data.
- 3 Compute

$$\mathbf{g} = \frac{1}{m} \nabla L(f(\mathbf{x}, \theta), y)$$

- 4 k th iteration,

$$\theta_{k+1} = \theta_k - \varepsilon_k \mathbf{g}$$

Notes: Converges if $\sum_{j=1}^{\infty} \varepsilon_j < \infty$, but tends to converge slowly (in high dimensions).

SGD with Momentum

Update SDG with the following changes:

- Choose an initial "velocity" vector $\mathbf{v}$.

SGD with Momentum

Update SDG with the following changes:

- Choose an initial "velocity" vector $\mathbf{v}$.
- Choose learning rate ε and momentum parameter α .

SGD with Momentum

Update SDG with the following changes:

- Choose an initial "velocity" vector $\mathbf{v}$.
- Choose learning rate ε and momentum parameter α .
- Update $\mathbf{v}$ by

$$\mathbf{v}_{k+1} = \alpha \mathbf{v}_k - \varepsilon \mathbf{g}$$

SGD with Momentum

Update SDG with the following changes:

- Choose an initial "velocity" vector $\mathbf{v}$.
- Choose learning rate ε and momentum parameter α .
- Update $\mathbf{v}$ by

$$\mathbf{v}_{k+1} = \alpha \mathbf{v}_k - \varepsilon \mathbf{g}$$

- Update θ by

$$\theta_{k+1} = \theta_k + \mathbf{v}_{k+1}$$

Adaptive Learning Rates

- AdaGrad
- RMSProp
- Adam

Goal: Keep the gradients out of local minima and exploring the parameter space efficiently.